

FICHE DE TD 3/3 • DIFFICILE

Informatique • 1re TI

Programmation JavaScript côté client

Module 2 : Programmation

Consignes : Les exercices suivants mobilisent l'ensemble des savoirs du chapitre. La rigueur algorithmique, la séparation HTML/JS et la gestion d'erreurs sont attendues. Aucun code HTML n'est fourni : vous devez concevoir la structure minimale nécessaire.

EXERCICE 1 Moteur de validation de formulaire dynamique

On souhaite concevoir un module JavaScript réutilisable `FormValidator` pour valider un formulaire d'inscription complexe côté client avant envoi au serveur. Le formulaire contient : identifiant (alphanumérique, 6–12 caractères), email (format standard), mot de passe (8+ caractères, 1 majuscule, 1 chiffre, 1 spécial), confirmation mot de passe, âge (entier 18–99), et une case à cocher CGU.

1. **Architecture :** Proposez une structure de données (objet littéral ou `new Object()`) pour stocker les règles de validation de chaque champ (sélecteur DOM, regex ou fonction de test, message d'erreur). Justifiez le choix d'un fichier `.js` externe chargé en `defer` plutôt que des scripts inline ou dans `<head>` sans attribut.
2. **Moteur générique :** Écrivez la fonction `validateField(fieldConfig)` qui prend une configuration de champ, récupère l'élément DOM correspondant (`getElementById / querySelector`), exécute le test, et retourne `true/false`. Elle doit afficher/masquer dynamiquement le message d'erreur dans un `<span>` adjacent (création/suppression DOM).
3. **Gestion événementielle :** Décrivez la stratégie d'attachement des écouteurs (`onChange`, `onFocus`, `onBlur`) pour valider au fil de l'eau (champ quitté) et à la soumission (`onClick` sur bouton `type="button"`). Pourquoi éviter `onSubmit` sur la balise `<form>` si l'on gère tout en JS ?
4. **Logique métier :** Implémentez la vérification de la force du mot de passe (fonction dédiée retournant un score 0–4) et la comparaison des deux champs mot de passe. Gérez le cas où l'utilisateur colle du texte (paste event non au programme : comment contourner avec `onChange` ?).
5. **Analyse critique :** Quelles sont les limites de sécurité de cette validation côté client ? Comment l'architecture proposée facilite-t-elle la maintenance (ajout/suppression de champs) par rapport à une suite de `if/else` imbriqués ?

EXERCICE 2 Système de gestion de stock « mono-page »

Une application web Single Page (sans rechargement) permet de gérer un stock de

pièces détachées. Les données résident dans un tableau d'objets `stock = []` en mémoire. Chaque pièce : `id` (entier), `ref` (chaîne « REF-XXXX »), `label`, `qty` (entier), `price` (float), `criticalThreshold` (entier).

1. **Initialisation** : À `onLoad`, le script charge un jeu de test (5 objets créés via `new Object()`) et appelle `renderTable()`. Écrivez `parseStockData(rawString)` qui transforme une chaîne CSV (ex : « 1;REF-001;Vis M6;150;0.15;10 ») en objet pièce, en gérant les conversions (`parseInt`, `parseFloat`) et les erreurs de format (retour `null`).
2. **Affichage dynamique** : `renderTable()` vide le `<tbody>` et le repeuple via `document.createElement / appendChild`. Les lignes dont `qty < criticalThreshold` portent une classe CSS `.alert`. Détaillez l'algorithme de parcours (boucle `for` ou `for...of`).
3. **Opérations CRUD simplifiées** :
 - `addPiece(formData)` : vérifie unicité `id/ref` (recherche séquentielle), ajoute au tableau, rafraîchit.
 - `updateQty(id, delta)` : incrémente/décrémente (entrée/sortie stock), empêche `qty < 0`, rafraîchit.
 - `deletePiece(id)` : confirmation `confirm()`, suppression dans le tableau (décalage ou `filter` non au programme : écrivez la suppression manuelle par décalage d'indices), rafraîchit.
4. **Statistiques** : Fonction `computeStats()` retournant un objet `{ totalItems, totalValue, nbCritical, avgPrice }`. Utilisez une seule boucle `for` (accumulateurs). Affichez les résultats dans un encart `<aside>` mis à jour après chaque opération.
5. **Persistance (réflexion)** : Le stockage en variable JS est volatile. Sans utiliser `localStorage` (hors programme), proposez une architecture pour simuler la persistance via un champ caché `<textarea id="export">` mis à jour en JSON (méthode `JSON.stringify` autorisée comme « boîte noire ») et un bouton « Importer » (`onChange + JSON.parse`). Quels risques de XSS ou d'injection lors de l'import ?

EXERCICE 3 Moteur de Quiz adaptatif à états

Concevoir un objet `QuizEngine` gérant un questionnaire à choix multiples (QCM) séquentiel. Données : tableau `questions[]` d'objets `{ id, text, choices[], correctIndex, weight }`. Un seul affichage à la fois (question courante), navigation Précédent/Suivant, soumission finale.

1. **Machine à états** : Identifiez les états internes nécessaires (`currentIndex`, `answers[]`, `started`, `finished`). Écrivez le constructeur `init(questions)` initialisant l'état et attachant les gestionnaires `onClick` sur les boutons `#next`, `#prev`, `#submit` (délégation d'événements sur un conteneur unique).

2. **Rendu conditionnel** : `render()` met à jour le DOM : numéro question, énoncé, liste radio `<input type="radio">` générée en boucle `for` sur `choices`. Pré-cocher la réponse si déjà enregistrée dans `answers[]`. Désactiver `#prev` si première question, masquer `#next` / afficher `#submit` si dernière.
3. **Logique de navigation** : `goTo(index)` valide l'index, sauvegarde la réponse courante (`onChange` sur les radios ou lecture au clic navigation), met à jour `currentIndex`, appelle `render()`. Gérer le cas où l'utilisateur clique « Suivant » sans choisir : bloquer ou autoriser (justifiez).
4. **Calcul du score pondéré** : `calculateScore()` parcourt `questions` et `answers` (boucle `for` unique), somme `weight` si `answers[i] === correctIndex`, retourne `{ raw, weighted, percent }`. Affichage final détaillé : tableau récapitulatif (Question, Votre réponse, Bonne réponse, Points).
5. **Accessibilité & Robustesse** : Comment garantir que `answers[]` reste synchronisé si l'utilisateur modifie une réponse via clavier (Tab/Espace) sans cliquer ? Proposez une solution utilisant `onChange` sur le conteneur des radios. Que se passe-t-il si `questions` est vide ou malformé à l'init ? (Gestion défensive).

EXERCICE 4 Analyse comparative & Refactoring « Legacy »

On vous confie le fichier `legacy.js` (ci-dessous) inclus via `<script src="legacy.js">` en fin de `<body>`. Il gère une calculatrice de TVA simple (HT → TTC, TTC → HT) avec historique.

```
// legacy.js
var taux = 20; var hist = [];
function calc() {
  var input = document.getElementById('montant').value;
  var type = document.getElementById('type').value;
  var res = 0;
  if (type == 'ht') { res = input * (1 + taux/100); }
  else if (type == 'ttc') { res = input / (1 + taux/100); }
  document.getElementById('resultat').innerHTML = res.toFixed(2);
  hist.push(input + ' ' + type + ' -> ' + res.toFixed(2));
  updateHist();
}
function updateHist() {
  var ul = document.getElementById('hist');
  ul.innerHTML = '';
  for (var i=0; i<hist.length; i++) {
    ul.innerHTML += '<li>' + hist[i] + '</li>'; // Concaténation répétée
  }
}
```

```
document.getElementById('btn').onclick = calc;  
document.getElementById('taux').onchange = function() { taux = parseInt(this.value) };
```

1. Critique globale (4 pts) : Identifiez 5 défauts majeurs relevant de : pollution de l'espace global (`var`), insertion HTML non sécurisée (`innerHTML +=`), typage faible / coercion implicite (`input * ...`), couplage fort HTML/JS (IDs en dur), absence de validation entrée (`parseFloat` manquant), performance DOM (réécriture complète historique), gestion événementielle unique (`onclick` écrase précédent).
2. Refactoring modulaire (4 pts) : Proposez une réécriture sous forme d'un module `const TVA = (() => { ... })()`; (IIFE) exposant `init(config)`. `config` contient les sélecteurs CSS. Utilisez `const/let`, `addEventListener` (bien que hors programme strict, on tolère la connaissance), `textContent` vs `innerHTML`, `DocumentFragment` pour l'historique, validation `parseFloat + isNaN`, séparation calcul / affichage.
3. Évènements & Portée (2 pts) : Expliquez pourquoi `var taux` dans l'IIFE ne pollue plus `window`. Comment accéder à l'historique depuis la console pour debug sans le rendre global? (Indice : closure / getter).
4. Insertion script (2 pts) : Comparez l'insertion actuelle (fin `<body>`) avec `<script src="..." defer>` dans `<head>`. Différences sur `DOMContentLoaded`, ordre d'exécution multiples scripts, blocage rendu. Quand `defer` est-il insuffisant (dépendance inter-scripts)?